

15-418 Assignment 3: Parallel VLSI Wire Routing Report

Jason (jasont2) Catherine (catheri5)

October x, 2026

1 Implementation Status

We first implemented a sequential correctness baseline before adding OpenMP parallelism. The starter code's compressed route representation was replaced with a keypoint representation: each wire stores up to five points, corresponding to the start point, up to three bends, and the end point. This matches the assignment checker format and makes route construction and validation easier to reason about.

During initialization, each input wire is assigned a simple legal route. If the endpoints share a row or column, the initial route is straight. Otherwise, the initial route is a horizontal-first one-bend path. The occupancy matrix is then constructed by adding each initial wire route.

The current baseline rerouting algorithm visits every wire for each iteration. For a wire being reconsidered, the current route is first removed from the occupancy matrix. The program then generates every legal route with at most three bends, scores each route using the sum of squared occupancy values that would result from adding the wire back, and chooses the lowest-cost route. Ties are broken by preserving the existing route. The selected route is then added back into the occupancy matrix.

2 Route Generation and Cost Calculation

For non-collinear endpoints, route generation currently includes horizontal-first and vertical-first routes with at most two bends, plus exactly three-bend routes through each strictly interior point in the endpoint bounding box. Choosing strictly interior points for the three-bend case prevents those routes from collapsing into duplicate one- or two-bend routes. Some at-most-two-bend routes do collapse to one-bend routes; consecutive duplicate keypoints are removed when constructing the wire.

When scoring a candidate route, the current wire has already been factored out of the occupancy matrix. Therefore each occupied cell on the candidate route is scored using $(occupancy[y][x] + 1)^2$. This computes the route’s relevant contribution if the current wire were added back along that path.

3 Within-Wires Parallelism

The first OpenMP implementation target is the within-wires strategy. For a single wire, all candidate routes can be scored independently while reading the same occupancy matrix. We plan to keep the outer loop over wires sequential at first, generate the candidate routes for the current wire, and parallelize the route-scoring loop when the number of routes is large enough to amortize OpenMP overhead.

Our current within-wires implementation uses per-thread local best routes during the parallel route-scoring loop. Each worker scores a subset of candidate routes and records its best local route and cost. After the parallel loop, the main thread reduces these per-thread results to choose the route for the current wire. This avoids locking inside the route-scoring loop.

We are comparing static and dynamic scheduling policies. Since different wires can have very different numbers of candidate routes, we also test a threshold below which route scoring remains sequential. The goal on the GHC machines is to expose enough route-scoring work to keep up to 8 threads busy. For the later PSC experiments, the same design will need to expose enough work to scale toward 128 threads.

3.1 Preliminary Within-Wires Tuning

We initially collected a small set of static and dynamic scheduling timings, but we removed those results from the report after discovering that the command-line thread count was parsed but not yet passed to OpenMP with `omp_set_num_threads(num_threads)`. Since those runs do not provide a valid comparison between 1-thread and 8-thread execution, we treat them only as a diagnostic step. The within-wires schedule and threshold experiments will be rerun after this fix.

Table ?? shows the first corrected static/dynamic comparison after applying the thread-count fix. These runs use 8 threads, $P = 0$, 5 iterations, a threshold multiplier of 8, and chunk size 8. Computation time still excludes initial route placement and occupancy initialization.

Input	Schedule	Init (s)	Compute (s)	Max Occ.	Cost
few wires	static	0.0217492720	1.2637013040	2	29236
medium wires	static	0.0276752920	4.6609008170	2	266415
few wires	dynamic	0.0221345980	1.2671972500	2	29236
medium wires	dynamic	0.0277004300	4.7129189950	2	266429

Table 1: Corrected preliminary within-wires static/dynamic comparison on GHC with $P = 0$. All runs passed validation.

Static scheduling is slightly faster in this comparison, but the difference is small. This suggests that schedule choice alone is not the dominant factor for the current within-wires implementation. The small cost difference on the medium input is likely due to schedule-dependent tie-breaking among equal-cost candidate routes.

4 Planned Across-Wires Parallelism

The across-wires strategy is more complicated because different threads make route decisions that update the shared occupancy matrix. Our initial plan is to follow the assignment’s batch model: a worker obtains a batch of wires, processes the wires in that batch sequentially, and then updates the occupancy matrix for the chosen routes. We will vary the batch size to study the tradeoff between synchronization overhead, staleness, load balance, and final routing quality.

Potential synchronization strategies include a single global lock, row-level locks, and tile-level locks. We expect the simplest correct version to use coarser synchronization first, then refine the locking granularity only if measurements show that update contention is a bottleneck.

5 Optimization Hypotheses

The main performance themes we plan to evaluate are workload balance, scheduling overhead, locality, and contention. Within-wires parallelism exposes candidate routes as independent tasks; this should work best for wires with large bounding boxes and many possible routes. Across-wires parallelism exposes wires or batches of wires as tasks; this may require dynamic scheduling or longest-task-first ordering to avoid stragglers.

We will also consider locality-aware across-wires batching as a later experiment. For example, wires could be grouped by bounding-box midpoint

or quadrant so that threads tend to work on nearby occupancy cells. This may improve cache locality, but it may also increase contention because nearby wires are more likely to overlap. We will only pursue this direction if cache miss or synchronization measurements suggest that the extra work is justified.

6 Experimental Plan

For the GHC experiments, we will collect results for 1, 2, 4, and 8 threads for both within-wires and across-wires implementations. We will report total time, computation time, speedup, cache misses, maximum occupancy, and total routing cost. After the GHC implementation is stable, we will run the required PSC scaling experiments for 1, 2, 4, 8, 16, 32, 64, and 128 threads.